

Tabla de contenido

Módulo 5 – Construir.....	2
Clase 7. Configuración, optimización y operación de procesos automatizados.....	2
Resultados de aprendizaje observables.....	2
Parte 1.....	2
Configuración segura y gestión de credenciales.....	2
Error handling, retries e idempotency.....	2
Observability y traceability.....	3
Parte 2.....	4
Evals de workflows con AI.....	4
Optimización y mejora continua.....	4
Gobernanza, cambio y continuidad.....	5
Casos de uso pertinentes.....	5
Limitaciones, riesgos, controversias y errores frecuentes.....	5
Buenas prácticas.....	5
Demostración sugerida para el docente.....	5
Preguntas de reflexión o comprobación.....	5
Ideas clave para convertir en diapositivas.....	6
Fuentes utilizadas en esta clase.....	6
Bibliografía.....	7

Módulo 5 – Construir

Clase 7. Configuración, optimización y operación de procesos automatizados

Propósito. Profundizar en confiabilidad, observabilidad, evaluación y mejora de automatizaciones con componentes de AI.

Resultados de aprendizaje observables

- Definir controles de errores, reintentos e idempotencia.
- Establecer métricas de proceso, calidad, costo y riesgo.
- Explicar supervisión humana según impacto.
- Proponer criterios de optimización sin sacrificar seguridad.

Pregunta orientadora: ¿Cómo mantener un flujo automatizado confiable cuando cambian datos, aplicaciones y modelos?

Parte 1

Configuración segura y gestión de credenciales

Explicación intuitiva. Configurar no es solo enlazar cuentas: exige identidad de servicio, permisos mínimos, secretos protegidos, ambientes y propietario. Las credenciales personales crean fragilidad y dificultan auditoría.

Explicación técnica. Técnicamente, usar OAuth o gestores de secretos, rotación y separación desarrollo–producción. Los tokens deben tener alcance y vencimiento adecuados. No deben aparecer en prompts, registros ni documentos.

Cuándo conviene y cuándo no. Conviene usar cuentas institucionales administradas. No conviene compartir claves por chat o incrustarlas en campos visibles.

Ejemplo, contraejemplo y simplificación docente. Correcto: conexión con permiso solo de lectura cuando basta. Error: administrador global para un flujo de notificación. Simplificación docente: cada llave abre solo la puerta necesaria.

Error handling, retries e idempotency

Explicación intuitiva. Los workflows fallan por red, límites, datos, credenciales o servicios. Retry ayuda ante fallas transitorias, pero repetir una action puede duplicar su efecto. Idempotency significa que procesar el mismo evento varias veces deja el mismo estado final.

Leer un registro puede repetirse con bajo riesgo; crear un pago o enviar una comunicación requiere protección contra duplicados.

Explicación técnica. Los errores se clasifican como transient, validation/business y permanent. Exponential backoff y jitter reducen carga en transient failures; invalid data debe ir a una exception queue, no a retries infinitos.

Idempotency keys, unique constraints y state machines detectan eventos ya procesados. Dead-letter queues conservan casos agotados. Compensating transactions revierten o neutralizan efectos cuando no existe atomicity.

Cuándo conviene y cuándo no. Conviene definir política por action y probar duplicados. No conviene retry indefinido ni tratar todos los errores como transitorios.

Una ruta manual debe conservar contexto suficiente para resolver sin exponer secretos.

Ejemplo correcto. Antes de crear una factura, comprobar el request ID; si existe, devolver el registro anterior.

Contraejemplo o error frecuente. Reintentar ``send_email`` cinco veces sin comprobar si el primer envío se completó.

Qué puede simplificar el docente. Puede usarse “antes de repetir, comprobar si ya ocurrió”. Deben conservarse categorías de error, backoff y dead-letter.

Observability y traceability

Explicación intuitiva. Observability permite responder qué pasó, dónde, cuándo y con qué versión. Traceability permite reconstruir el camino de un caso. Sin ellas, una automation es una caja negra que solo avisa “falló”.

Registrar todo tampoco es correcto: prompts y tool outputs pueden contener datos personales o secretos. Se necesita evidencia operativa mínima y protegida.

Explicación técnica. Logs registran eventos; metrics agregan tendencias; traces conectan steps mediante correlation IDs. Campos útiles incluyen workflow/version, event ID, step, status, latency, model/prompt version, validation result y approval.

Sensitive payloads deben redactarse, tokenizarse o excluirse. Retention, access control y audit log aplican también a la telemetría. Alerts deben corresponder a acciones operables, no a cada variación.

Cuándo conviene y cuándo no. Conviene instrumentar desde el piloto y definir owner de alertas. No conviene registrar contenido completo por defecto ni medir solo disponibilidad cuando el riesgo es calidad del output.

Un dashboard debe combinar salud técnica, calidad, costo y riesgo.

Ejemplo correcto. Registrar ID, versión, resultado de validación y escalamiento; guardar el documento original en un repositorio autorizado, no duplicarlo en logs.

Contraejemplo o error frecuente. Escribir prompts con credenciales en logs o conservarlos indefinidamente para depuración.

Qué puede simplificar el docente. Puede usarse tablero de vuelo. Deben conservarse logs–metrics–traces, correlation ID y minimización.

Parte 2

Evals de workflows con AI

Explicación intuitiva. Un workflow con AI debe evaluarse como proceso y como componente probabilístico. Puede mejorar clasificación y empeorar tiempo total por revisiones; puede ahorrar minutos y aumentar errores críticos. Calidad, velocidad, costo y riesgo deben observarse juntos.

La baseline muestra qué ocurre sin la nueva solución. Sin ella no se sabe si el workflow mejora el trabajo o solo produce una experiencia más novedosa.

Explicación técnica. Las métricas pueden incluir precision/recall, coverage, escalation rate, critical error rate, end-to-end latency, cost per case, rework y user outcome. Deben segmentarse por tipo de caso y volumen.

Prompt, model, retrieval source y workflow version deben registrarse. Cada cambio requiere regression eval. Drift puede provenir de input distribution, documentos, proveedor o comportamiento humano.

Online monitoring detecta tendencias; offline evals comparan versiones sobre un conjunto estable. Sampling de casos reales debe respetar privacidad y evitar que un reviewer vea información innecesaria.

Cuándo conviene y cuándo no. Conviene desplegar con thresholds, human route y rollback. No conviene aprobar un cambio porque cinco ejemplos mejoraron ni optimizar tokens sacrificando calidad.

Fallas críticas deben analizarse individualmente aunque sean raras.

Ejemplo correcto. Comparar contra baseline manual en 100 casos estratificados y medir error crítico, escalamiento, tiempo y costo.

Contraejemplo o error frecuente. Reportar solo porcentaje de automatización o ahorro de tokens.

Qué puede simplificar el docente. Puede mostrarse calidad + tiempo + costo + riesgo. Deben conservarse baseline, segmentación, versioning y regression eval.

Optimización y mejora continua

Explicación intuitiva. Optimizar consiste en eliminar pasos innecesarios, reducir llamadas, mejorar recuperación, usar modelos adecuados y ajustar umbrales. Debe proteger exactitud, seguridad y experiencia.

Explicación técnica. Técnicamente, observar cuellos de botella y errores antes de cambiar. Caché, lotes y modelos más pequeños pueden reducir costo; rutas condicionales reservan modelos avanzados para casos complejos.

Cuándo conviene y cuándo no. Conviene optimizar después de obtener línea base. No conviene reducir revisión en casos críticos solo para disminuir tiempo.

Ejemplo, contraejemplo y simplificación docente. Correcto: ruta simple para casos claros y escalamiento para ambiguos. Error: retirar controles para “acelerar”. Simplificación docente: primero correcto y observable; luego eficiente.

Gobernanza, cambio y continuidad

Explicación intuitiva. Los flujos necesitan dueño, documentación, revisión de accesos, plan de cambios, respaldo y procedimiento manual. Los proveedores y conectores cambian; una automatización sin mantenimiento acumula riesgo.

Explicación técnica. Técnicamente, versionar configuración, definir RTO/RPO cuando aplique, probar restauración y mantener inventario de dependencias. Marcos de gestión como ISO/IEC 42 001 ayudan a institucionalizar responsabilidades [28].

Cuándo conviene y cuándo no. Conviene establecer cadencia de revisión proporcional al impacto. No conviene depender de una sola persona o plataforma sin exportación.

Ejemplo, contraejemplo y simplificación docente. Correcto: runbook, contacto y desactivación. Error: flujo crítico sin documentación. Simplificación docente: todo robot necesita propietario, tablero y freno.

Casos de uso pertinentes

- Monitorear solicitudes pendientes y alertar excepciones.
- Evaluar periódicamente calidad del componente generativo.
- Optimizar rutas por complejidad manteniendo controles.

Limitaciones, riesgos, controversias y errores frecuentes

- Reintentos que multiplican efectos.
- Logs que filtran datos sensibles.
- Deriva por cambios de modelo o fuente.
- Optimización local que empeora el proceso completo.

Buenas prácticas

- Definir SLO o expectativas operativas.
- Probar fallas y reversión, no solo éxito.
- Versionar prompts, flujos y fuentes.
- Revisar permisos y dependencias periódicamente.

Demostración sugerida para el docente

Ejecutar un flujo con un evento duplicado y una respuesta inválida de AI; mostrar cómo ID, validación, ruta humana y registro evitan efectos incorrectos. Usar datos sintéticos.

Preguntas de reflexión o comprobación

- ¿Qué debe reintentarse y qué no?
- ¿Qué dato mínimo necesita el log?
- ¿Cómo detectar cambio de calidad?

- ¿Qué procedimiento manual existe?

Ideas clave para convertir en diapositivas

- Confiabilidad: error, reintento, idempotencia.
- Observabilidad sin sobreexponer datos.
- Métricas equilibradas.
- Regresión después de cada cambio.
- Dueño, documentación y freno.

Fuentes utilizadas en esta clase

Referencias principales: [19], [26], [27], [28], [29], [30]. Las características de productos citadas deben verificarse de nuevo antes de impartir la sesión, porque pueden cambiar con versiones, planes y región.

Bibliografía

La bibliografía completa se encuentra en el siguiente archivo:

<https://evinracher.github.io/curso-sinergia/recursos/fundamentos-teoricos.pdf>